

# Unsupervised Learning

Qian Wang

School of Biomedical Engineering  
ShanghaiTech University

# Types of Learning

---

- According to the types of feedback that can accompany the inputs, there are three main types of learning.
  - **Supervised learning:**
    - The agent observes input-output pairs and learns a function that maps from input to output.
  - **Unsupervised learning:**
    - The agent learns patterns in the input without any explicit feedback.
  - **Reinforcement learning:**
    - The agent learns from a series of reinforcements: rewards and punishments.

# Types of Learning

---

|                   | <i>Supervised Learning</i>       | <i>Unsupervised Learning</i> |
|-------------------|----------------------------------|------------------------------|
| <i>Discrete</i>   | classification or categorization | clustering                   |
| <i>Continuous</i> | regression                       | dimensionality reduction     |

# Types of Learning

---

- **Classification** involves predicting discrete labels or categories for new observations based on a training dataset with known labels.
- **Regression** is the process of predicting a continuous numeric value for new observations based on a training dataset with known numeric outcomes.

# Types of Learning

---

- **Clustering** is the technique of grouping a set of objects in such a way that objects in the same group (cluster) are more similar to each other than to those in other groups.
- **Dimensionality reduction** is the process of reducing the number of features under consideration, by obtaining a set of principal features or lower-dimensional representations.

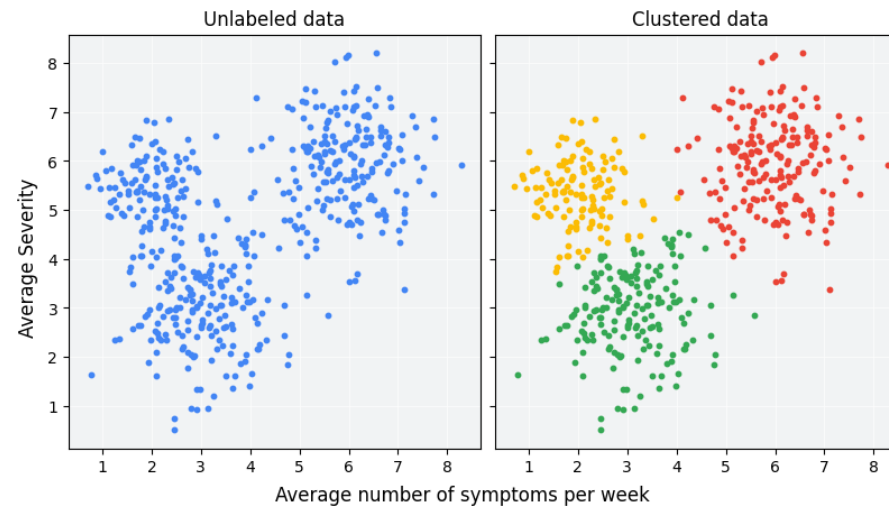
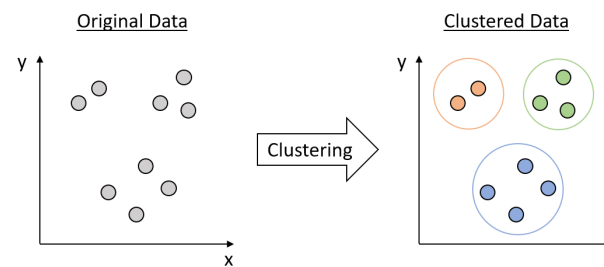
# Why Do We Cluster?

---

- Summarizing Data: Clustering helps in managing and interpreting large datasets by grouping similar data points together. This can make it easier to get an overview of the data and identify patterns.
- Compression and Denoising: By grouping similar data patches, clustering can be used for compressing data and removing noise, making data storage and transmission more efficient.
- Prediction: Clustering can enhance predictive modeling by assuming that data points within the same cluster are likely to share similar labels, aiding in tasks like classification and regression.
- Segmentation: In image processing, clustering can separate an image into different regions, making it easier to analyze and interpret different parts of the image.

# Why Do We Cluster?

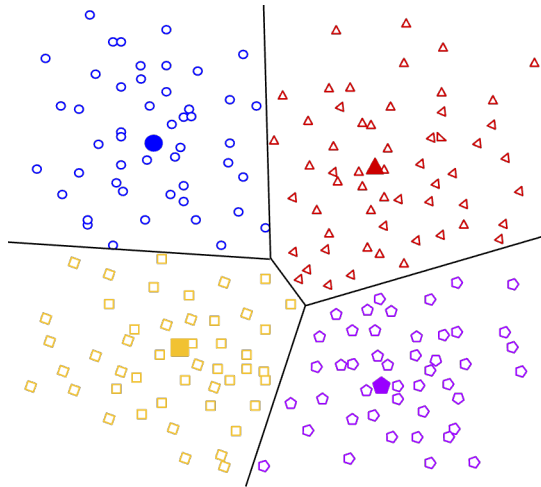
---



# Centroid-based Clustering

---

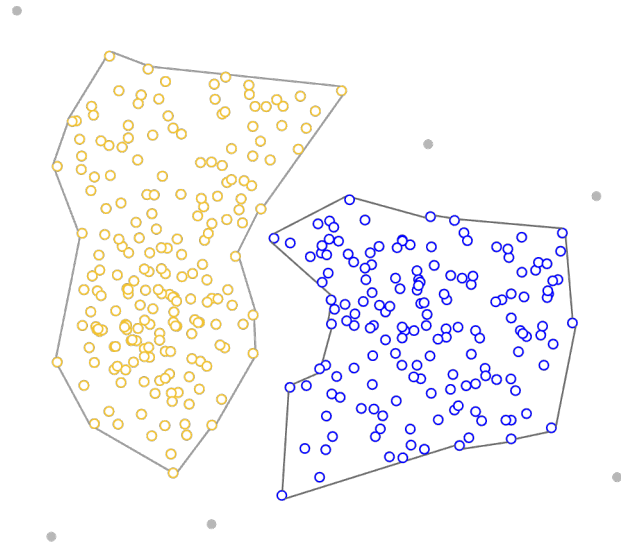
- Centroid-based clustering organizes the data into non-hierarchical clusters.
  - The centroid of a cluster is the arithmetic mean of all the points in the cluster.



# Density-based Clustering

---

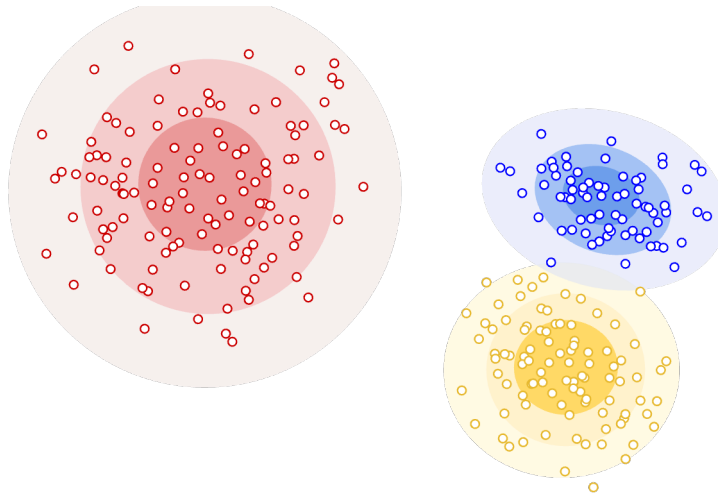
- Density-based clustering connects contiguous areas of high example density into clusters.
  - This allows for the discovery of any number of clusters of any shape.
  - Outliers are not assigned to clusters.



# Distribution-based Clustering

---

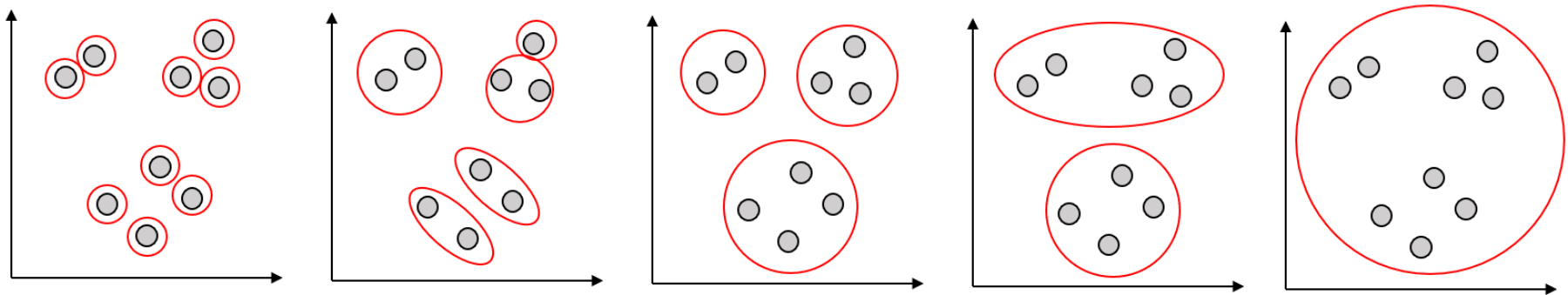
- This clustering approach assumes data is composed of probabilistic distributions, such as Gaussian distributions.
  - As distance from the distribution's center increases, the probability that a point belongs to the distribution decreases.



# Hierarchical Clustering

---

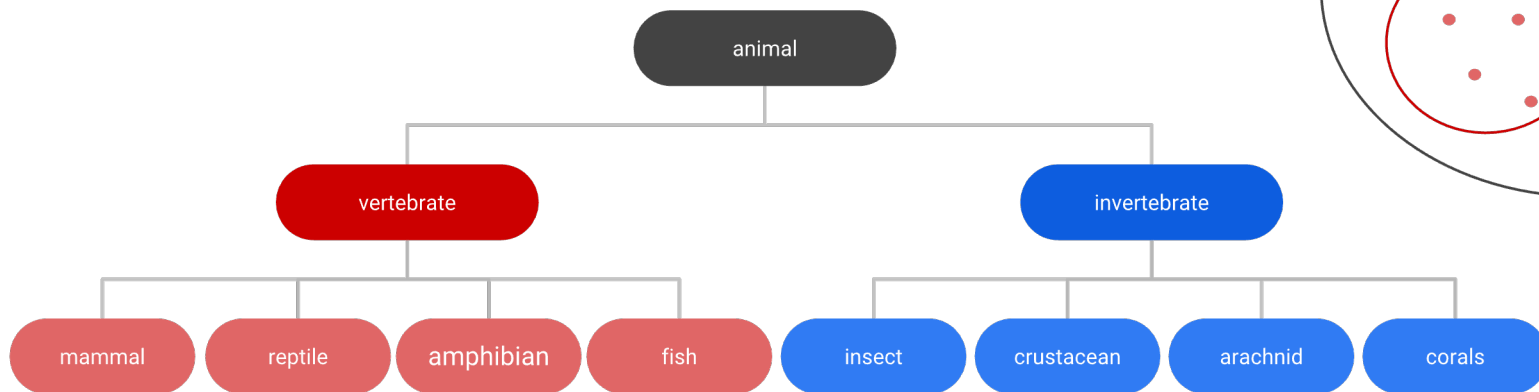
- Hierarchical clustering creates a tree of clusters.
  - It is well suited to hierarchical data, such as taxonomies.



# Hierarchical Clustering

---

- Hierarchical clustering creates a tree of clusters.
  - It is well suited to hierarchical data, such as taxonomies.



# K-means Clustering Method

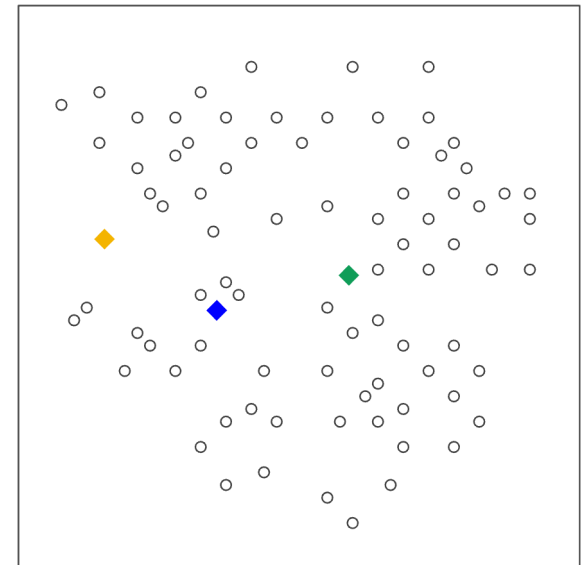
---

- K-means is the most popular clustering method.
- The goal of **k-means clustering** is to divide the data into  $k$  clusters where each data point belongs to the cluster with the nearest mean, achieving **intra-cluster similarity** while maximizing **inter-cluster dissimilarity**.
- Advantages:
  - Simplicity and Efficiency: K-means is relatively simple to implement and computationally efficient.
  - Scalability: It can be scaled to work with large datasets.

# K-means: Steps

---

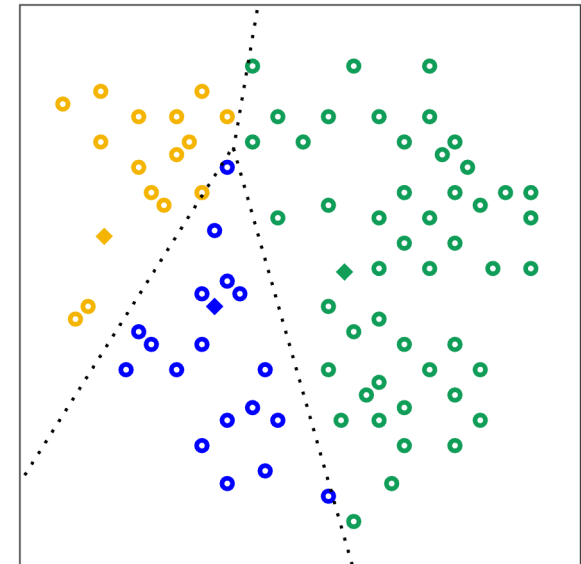
1. Provide an initial guess for  $k$  (e.g.,  $k=3$ );
2. Randomly choose  $k$  centroids.



# K-means: Steps

---

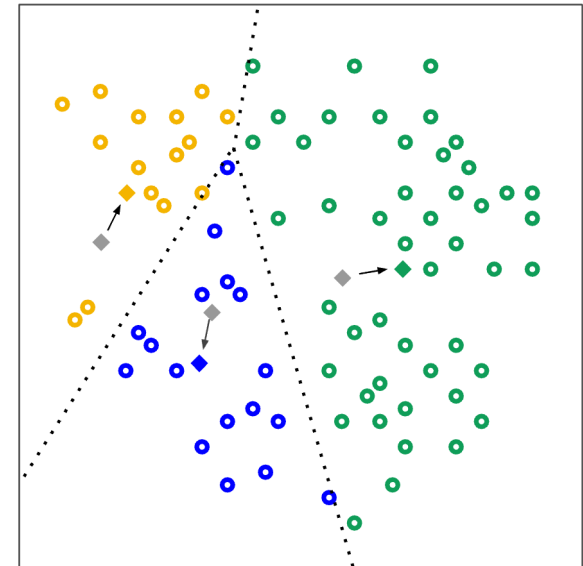
1. Provide an initial guess for  $k$  (e.g.,  $k=3$ );
2. Randomly choose  $k$  centroids;
3. Assign each point to the nearest centroid to get  $k$  initial clusters;



# K-means: Steps

---

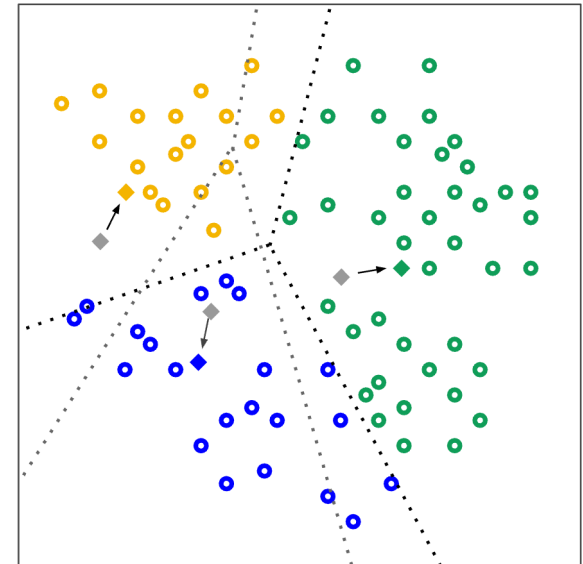
1. Provide an initial guess for  $k$  (e.g.,  $k=3$ );
2. Randomly choose  $k$  centroids;
3. Assign each point to the nearest centroid to get  $k$  initial clusters;
4. For each cluster, calculate a new centroid by taking the mean position of all points in the cluster.



# K-means: Steps

---

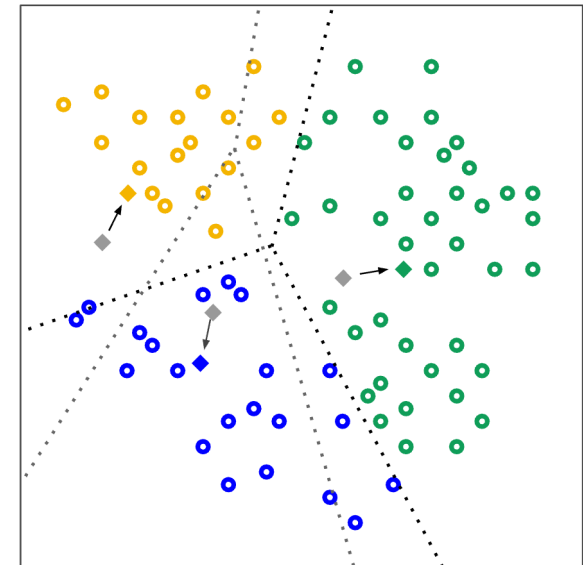
1. Provide an initial guess for  $k$  (e.g.,  $k=3$ );
2. Randomly choose  $k$  centroids;
3. Assign each point to the nearest centroid to get  $k$  initial clusters;
4. For each cluster, calculate a new centroid by taking the mean position of all points in the cluster;
5. Reassign each point to the nearest new centroid;



# K-means: Steps

---

1. Provide an initial guess for  $k$  (e.g.,  $k=3$ );
2. Randomly choose  $k$  centroids;
3. Assign each point to the nearest centroid to get  $k$  initial clusters;
4. For each cluster, calculate a new centroid by taking the mean position of all points in the cluster;
5. Reassign each point to the nearest new centroid;
6. Repeat steps 4 and 5, recalculating centroids and cluster membership, until points no longer change clusters.



# K-means: Implementation (Scikit-Learn)

---

```
class sklearn.cluster.KMeans(n_clusters=8, *, init='k-means++',
n_init='auto', max_iter=300, tol=0.0001, verbose=0, random_state=None,
copy_x=True, algorithm='lloyd')
```

## Examples

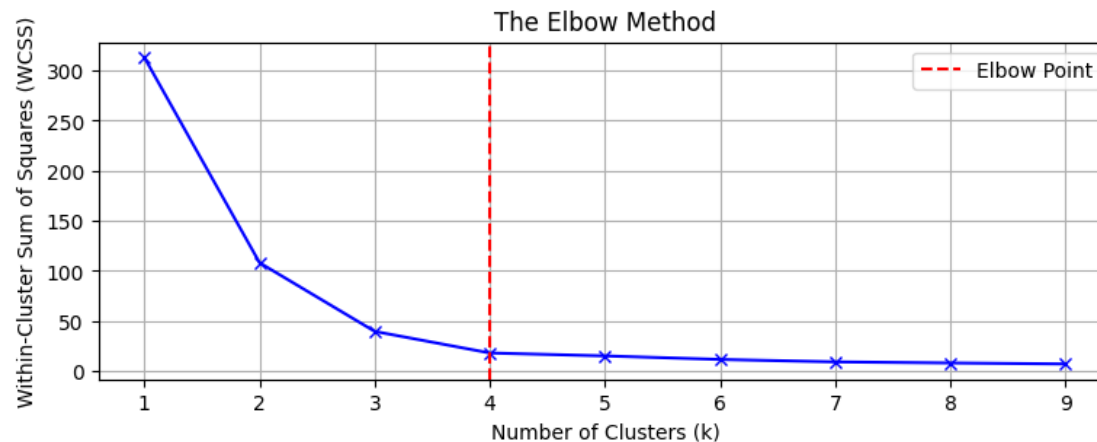
```
>>> from sklearn.cluster import KMeans
>>> import numpy as np
>>> X = np.array([[1, 2], [1, 4], [1, 0],
...              [10, 2], [10, 4], [10, 0]])
>>> kmeans = KMeans(n_clusters=2, random_state=0, n_init="auto").fit(X)
>>> kmeans.labels_
array([1, 1, 1, 0, 0, 0], dtype=int32)
>>> kmeans.predict([[0, 0], [12, 3]])
array([1, 0], dtype=int32)
>>> kmeans.cluster_centers_
array([[10.,  2.],
       [ 1.,  2.]])
```

<https://scikit-learn.org/1.5/modules/generated/sklearn.cluster.KMeans.html>

# K-means: Considerations

---

- Number of clusters ( $k$ ): The performance of  $k$ -means heavily depends on the choice of  $k$ .
  - Within-Cluster Sum of Squares (WCSS): defined as sum of the squared distances between each point and its cluster centroid, and measures how well the data points are clustered around their respective centroids.



This “elbow” point suggests the optimal number of clusters - adding more clusters (beyond this point) yields diminishing returns in WCSS.

# K-means Clustering: Considerations

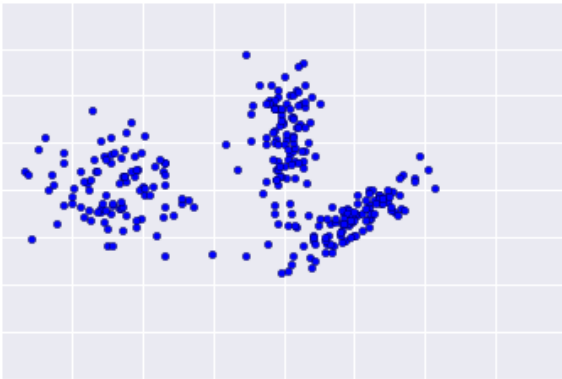
---

- Initial centroids: Random initialization may lead to different results; hence, multiple runs with different initializations can enhance robustness.
- Cluster shape and size: k-means assumes clusters to be spherical and of similar size, which may not always be the case in real-world data.
  - Why k-means has this assumption?

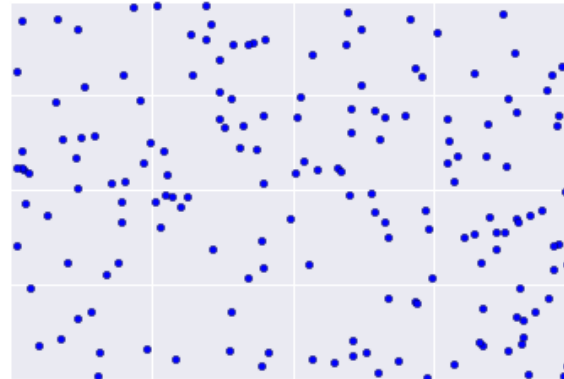
# Evaluating Clustering Results

---

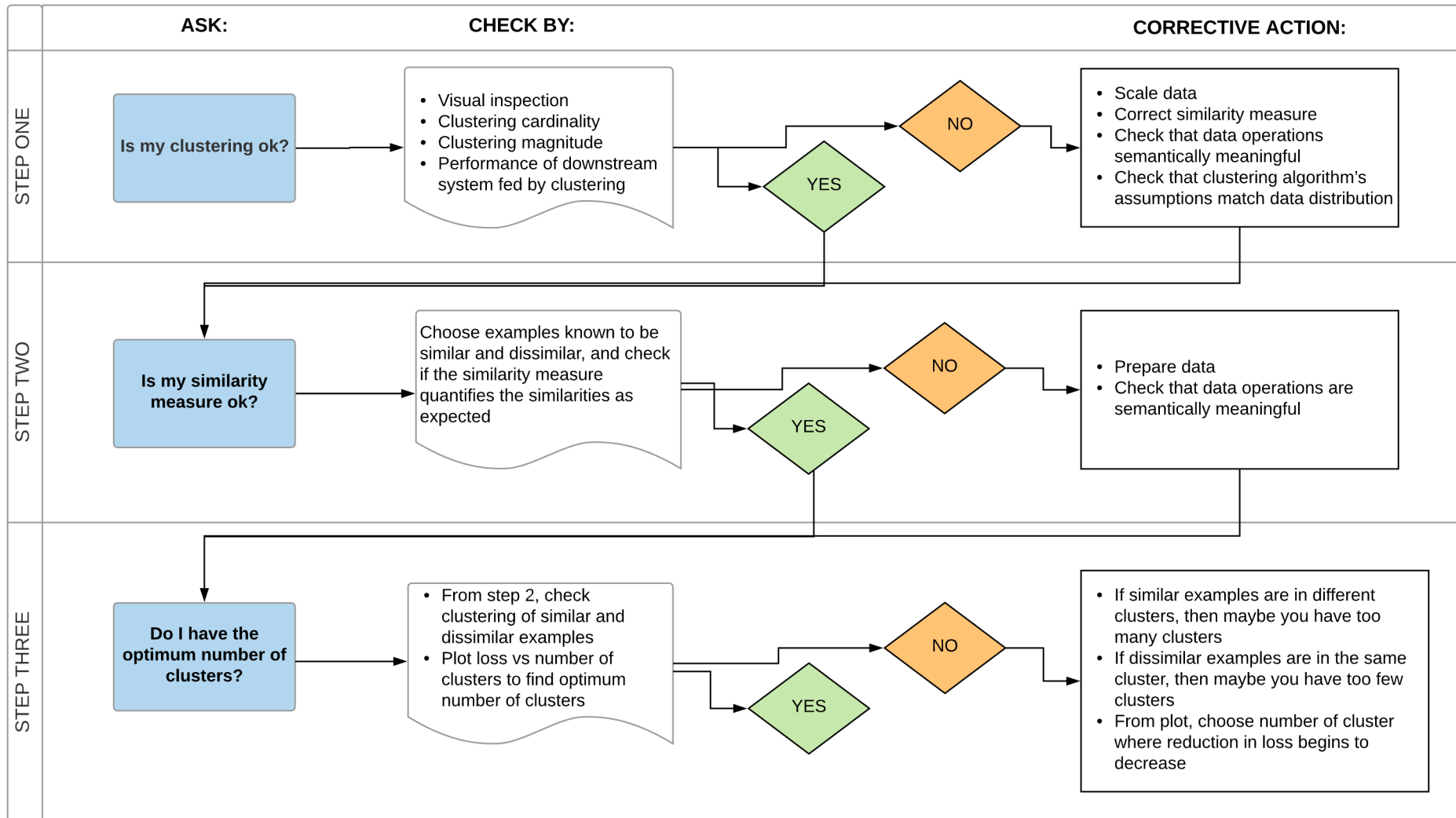
- Because clustering is unsupervised, no ground truth is available to verify results.
- The absence of truth complicates assessment of clustering quality.



An ideal data plot

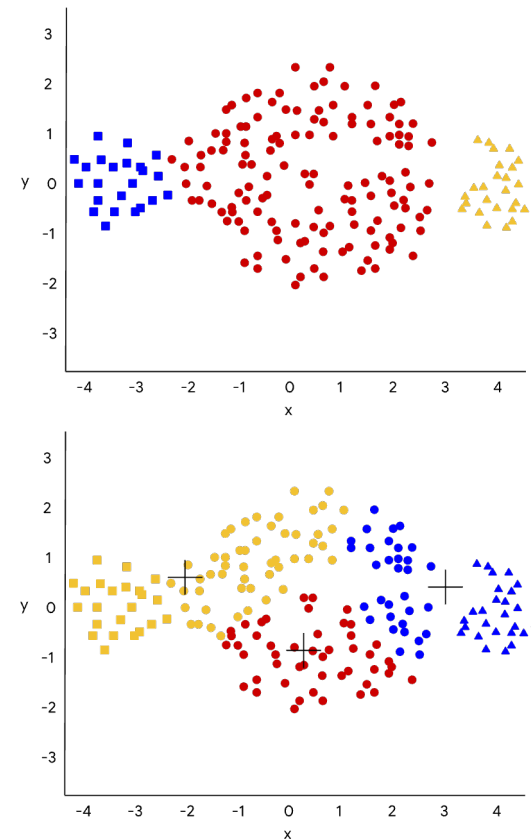


A more realistic data plot



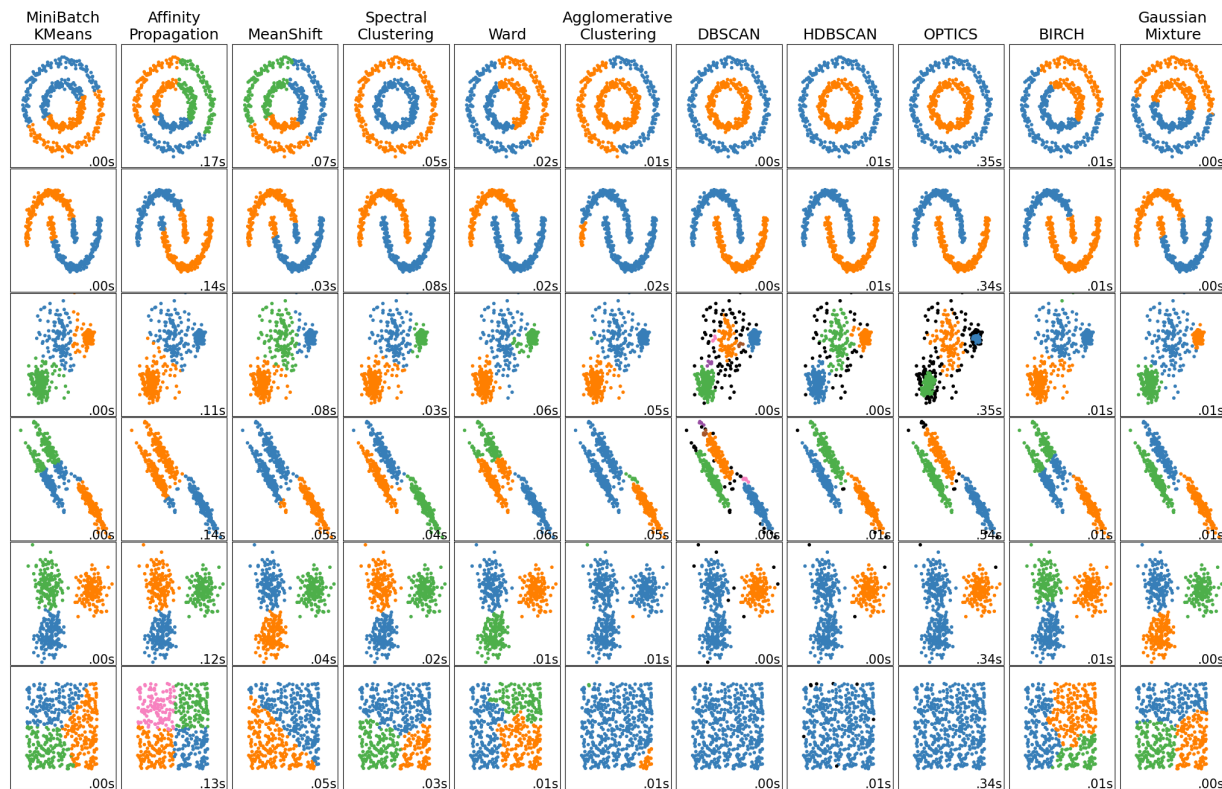
# K-means: Limitations

- $K$  must be chosen manually;
- Results depend on initial values;
- Difficulty clustering data of varying sizes and densities;
- Difficulty clustering outliers;
- Difficulty scaling with number of dimensions.



# Comparing Clustering Methods

---

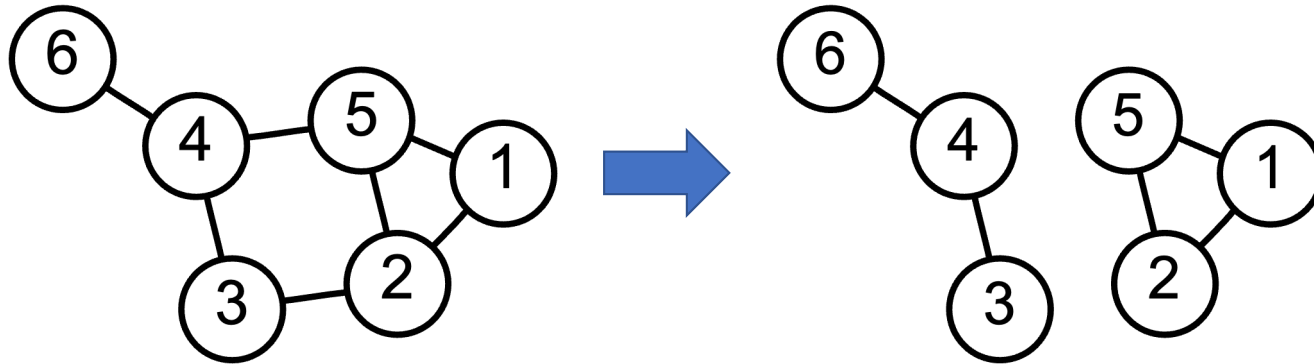


[https://scikit-learn.org/1.5/auto\\_examples/cluster/plot\\_cluster\\_comparison.html](https://scikit-learn.org/1.5/auto_examples/cluster/plot_cluster_comparison.html)

# Spectral Clustering

---

- In spectral clustering, we use a graph to describe the mutual similarity of the data.
- By using the spectrum (eigenvalues) of the similarity matrix of the graph, we can implement clustering to separate all data into several groups.



# Spectral Clustering: Conception

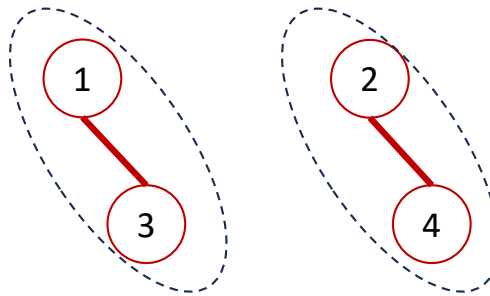
---

- Similarity matrix (also known as an affinity matrix)  $A$ :
  - Each entry  $A_{ij}$  indicates the similarity between data points  $i$  and  $j$ ;
  - Common choices include Gaussian (RBF) kernel or k-nearest neighbors to define similarities.
- **Graph Laplacian**  $L = D - A$ :
  - $D$  is the degree matrix ( $D_{ii}$  is the sum of similarities for a data point);
  - $L$  captures the structure of the similarity graph

# Spectral Clustering

---

- Let's do a practice:
  - What is  $A$ ,  $D$ , and  $L$ ?
  - What will happen if we perform eigendecomposition to  $L$ ?



If there are  $k$  connected components in the graph, the graph Laplacian matrix  $L$  will have  $k$  eigenvalues that are zero (in real case, it means eigenvalues that are very close to zero).

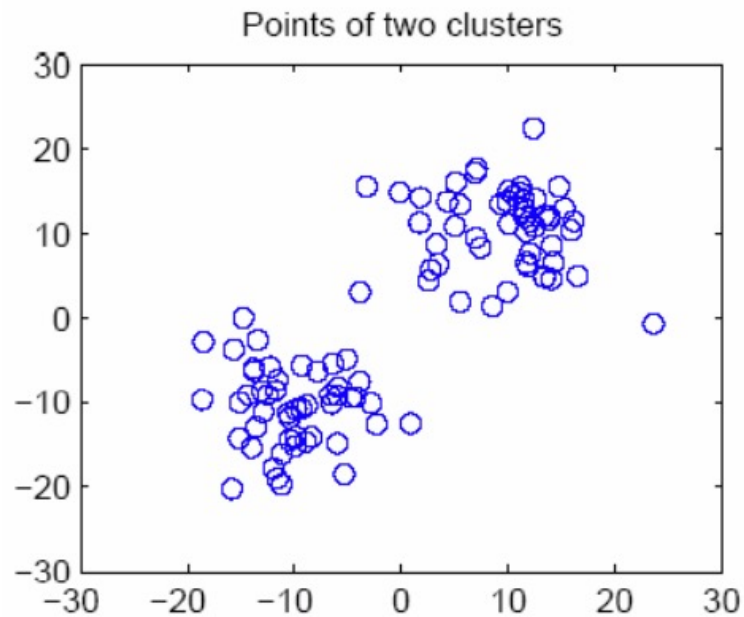
# Spectral Clustering: Steps

---

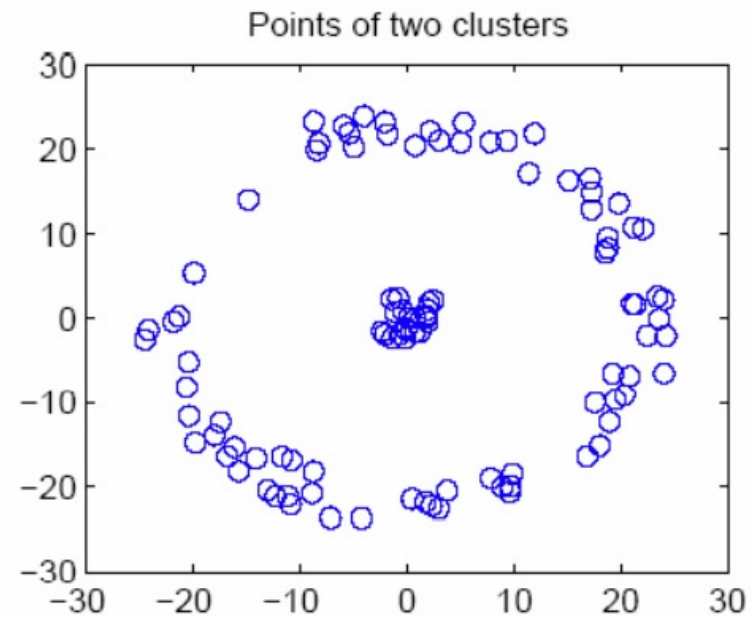
1. Construct the similarity graph  $A$ ;
2. Compute the Laplacian matrix  $L$ ;
3. Perform eigen-decomposition to  $L$ , and identify the  $k$  smallest eigenvalues and their corresponding eigenvectors;
4. Project each data point to the selected eigenvectors corresponding to the  $k$  smallest eigenvalues, and derive a new  $k$ -dimensional feature representation for the data point;
5. Apply a standard clustering algorithm (e.g., k-means) to partition the data points into  $k$  clusters.

# Spectral Clustering vs. K-means

---



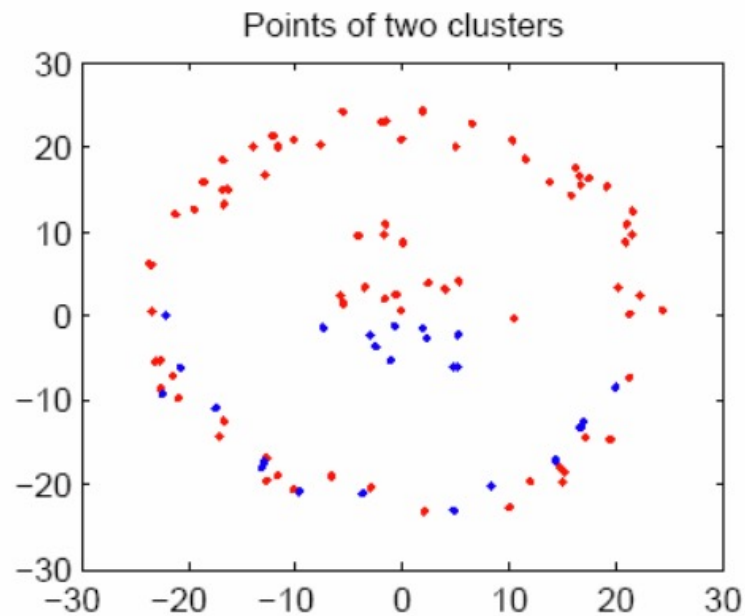
Both perform same



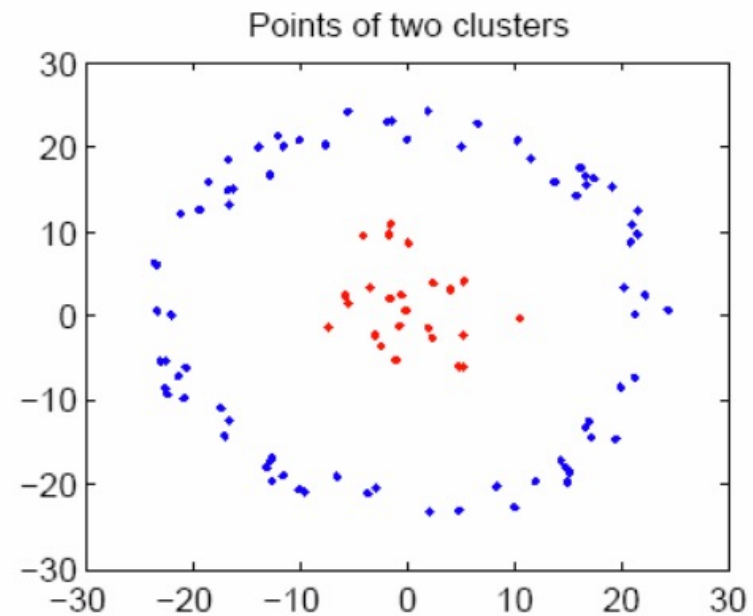
Spectral clustering is superior

# Spectral Clustering vs. K-means

---



k-means output



Spectral clustering output

# Dimensionality Reduction

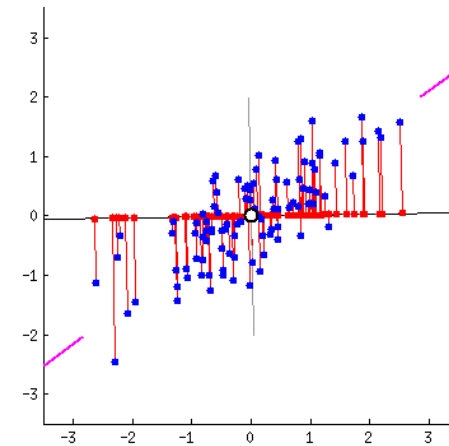
---

- Dimension reduction is a vital concept in data science and machine learning.
  - Feature selection: This involves selecting a subset of the original features without altering their values.
    - Filter methods: Use statistical techniques to select features with the highest relevance to the target variable, such as Chi-square tests, correlation coefficients, or mutual information.
    - Wrapper methods: Utilize predictive models to evaluate feature subsets' performance, typically through techniques like recursive feature elimination (RFE).
    - Embedded methods: Integrate the feature selection process into the training of a predictive model, examples include LASSO (Least Absolute Shrinkage and Selection Operator) which imposes a penalty on the coefficient size.
  - Feature extraction: This involves transforming the original features into a new set of features. We will focus on feature extraction next.

# Principal Component Analysis (PCA)

---

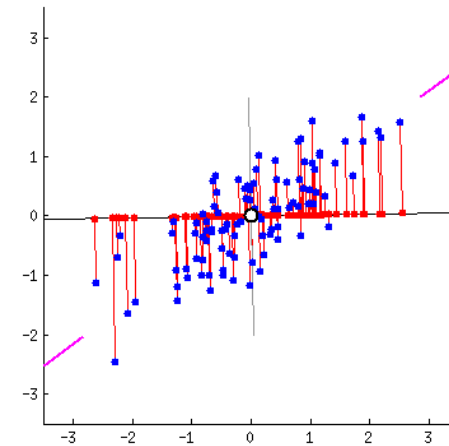
- PCA converts original correlated features into a set of uncorrelated principal components, ranked by the amount of variance they capture from the original data.
  - PCA relies on the eigenvalue decomposition of the covariance matrix.
  - The eigenvalues indicate the amount of variance captured in the direction of the corresponding eigenvector.



# Principal Component Analysis (PCA)

---

- PCA converts original correlated features into a set of uncorrelated principal components, ranked by the amount of variance they capture from the original data.
  - We define the first principal component to capture the largest amount of variance.
  - Each subsequent component captures the remaining variance under the constraint that it is orthogonal (uncorrelated) to the previous components.
  - By selecting the top  $k$  principal components, you can construct a lower-dimensional representation of the data that retains most of the original information.



# PCA: Steps

---

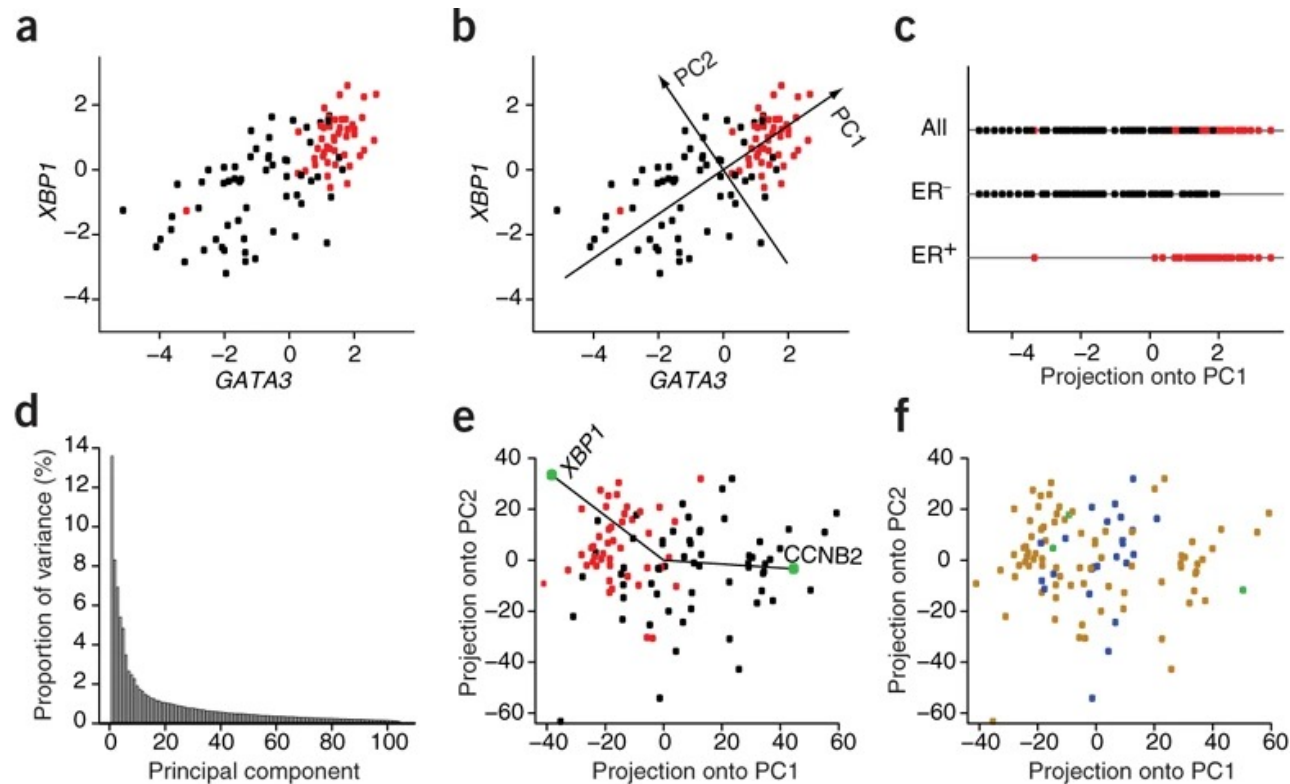
1. Standardize the Data: Since PCA is sensitive to the scale of the data, it is important to standardize the features so that they have a mean of 0 and a standard deviation of 1.
2. Compute the Covariance Matrix: This matrix represents how the features in the data are correlated with each other.
3. Calculate Eigenvalues and Eigenvectors: The eigenvalues represent the magnitude of variance captured by each principal component, and the eigenvectors point in the direction of the maximum variance.

## PCA: Steps

---

4. Sort Eigenvalues and Eigenvectors: The eigenvalues and their corresponding eigenvectors are sorted in descending order based on the eigenvalues. This sorting helps identify the principal components that capture the most variance.
5. Select Principal Components: Choose the top  $k$  eigenvectors (principal components) based on the sorted eigenvalues. These components form the new feature space.
6. Transform the Data: Project the original data onto the new feature space defined by the selected principal components.

# PCA: Example



(a) Each dot represents a breast cancer sample plotted against its expression levels for two genes. (In **a–c, e**, samples are colored according to estrogen receptor (ER) status: ER<sup>+</sup>, red; ER<sup>-</sup>, black). (b) PCA identifies the two directions (PC1 and PC2) along which the data have the largest spread. (c) Samples plotted in one dimension using their projections onto the first principal component (PC1) for ER<sup>+</sup>, ER<sup>-</sup> and all samples separately. (d) The variance of the principal components when PCA is applied to all 8,534 genes with expression levels for all samples...

<https://www.nature.com/articles/nbt0308-303>

# Linear Discriminant Analysis (LDA)

---

- LDA is primarily used for classification problems. It maximizes the ratio of between-class variance to within-class variance, creating a new axis that provides the best separation between different classes.

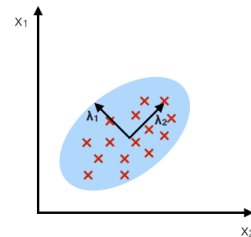
# LDA vs. PCA

---

- While both LDA and PCA are used for dimensionality reduction, PCA is unsupervised and focuses on maximizing variance without considering class labels.
- In contrast, LDA is supervised and seeks to maximize class separation.

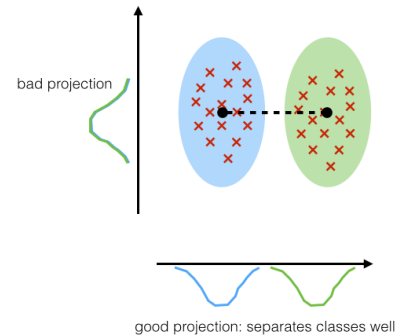
## PCA:

component axes that maximize the variance



## LDA:

maximizing the component axes for class-separation



# t-Distributed Stochastic Neighbor Embedding

---

- t-SNE is short for t-Distributed Stochastic Neighbor Embedding.
- It is a nonlinear technique particularly useful for visualizing high-dimensional data.
  - High-dimensional space to low-dimensional space mapping: t-SNE maps high-dimensional data to a lower-dimensional space (usually 2D or 3D).
  - Preserving local structure: t-SNE focuses on preserving the local structure of data, by minimizing the divergence between two probability distributions: one representing pairwise similarities of the input objects in the high-dimensional space and one representing pairwise similarities of the corresponding low-dimensional points.

# Summary: Machine Learning

---

- Machine learning: supervised learning, unsupervised learning.
- Supervised learning:
  - Training – testing (inference) paradigm.
  - Classification, regression.
  - K-NN, SVM, decision tree, bagging (random forest), boosting.
- Unsupervised learning:
  - Clustering: k-means, spectral clustering.
  - Dimensionality reduction: PCA, t-SNE.
- Applications?

Thank You!